ParkOps D2: Design

EECS 3311 B, Fall 2026 · Deliverable 2: a working supervised agent · Route A (ParkOps) · Bryan Maristanez · November 2026

1. Scope

D2 turns D1's proposal path into a supervised agent for the supplied fixture. `StudentApplication.run(ModelProvider, int)` runs a bounded model/tool loop that can read booking B1, list eligible spaces and create one `PENDING` proposal, and then stops. Separate operator commands (`approve`, `reject`, `execute`) decide what happens next; `execute` re-checks the current state and either commits once or returns a named refusal. `tick(long)` detects bookings on closed spaces with caller-supplied time. Policy version 1 stays in force. `bash run.sh run scripted` demonstrates every path with replayed responses (not a live model run); `bash run.sh run live` uses the same loop with the supplied Ollama adapter. Figures 1–5 follow the text.

2. Components and dependencies

The code is split into the four kinds D2.4 names. Dependencies point inward: the model protocol and the domain know nothing about coordination or storage, and the domain imports nothing outside itself.

Kind	Classes	Responsibility	Depends on
Model protocol	`Tool`, `ToolRequest`, `ToolRequestParser`	Turn untrusted reply text into a checked request or a rejection reason: exactly three string properties, one of four tools, goal booking B1, an empty `targetId` except for a proposal naming a known space. Never runs anything.	`MiniJson` (starter); the goal id and space ids are passed in, so it never reads storage
Coordination	`StudentApplication`, `AgentRunner`, `PeriodicChecker`	`StudentApplication` is the only entry point: queries, `propose`, operator commands and `tick`. `AgentRunner` owns one conversation: call limit, history, invalid count, read → list → propose order, stale-version checks and the one-active-proposal rule. `PeriodicChecker` owns tick time and duplicate suppression.	Protocol, domain, storage; `ModelProvider`
Domain rules	`Booking`, `Space`, `Proposal`, `ProposalStatus`, `EligibilityPolicy`, `ExecutionOutcome`, `DomainRules`	Validation, the policy-1 rule, legal status transitions (`approve/reject` only from `PENDING`, `markExecuted` only from `APPROVED`), and controlled changes: `Booking.movedTo` and `withDuration` return copies with version + 1; `Space` changes only through `occupy`, `release`, `close`. `ExecutionOutcome` names the five execution results (Figure 1 draws it with the values returned to callers).	Nothing outside the domain
Data storage	`ParkingStore`, `CommitRecord`, `Notification`	Holds the booking, spaces, proposals and the commit and simulated-notification records for one application. Stores and finds; no rules.	Domain
Provider boundary (starter)	`ModelProvider`, `ReplayProvider`, `OllamaProvider`, `ChatMessage`	`ReplayProvider` is the deterministic test double; `OllamaProvider` is the real route. Both return untrusted text through the same interface, so replay and live share one loop.	`MiniJson`

Callers only ever receive copies: snapshot maps, `ProposalView`, `AgentResult` (its transcript is copied), and immutable records. `StudentApplication` keeps its `ParkingStore` and `PeriodicChecker` in private fields and never returns them or the domain objects they hold, so every change goes through its checked methods.

3. Proposal lifecycle

Figure 5 draws these transitions. `execute` checks, in order: the proposal exists and is for B1; already `EXECUTED` → `DUPLICATE`; not `APPROVED` → `NOT_APPROVED`; booking or policy version differs from the recorded one → `STALE`; target not eligible now → `UNAVAILABLE_OR_INELIGIBLE`. Only when all pass does `commit` occupy the target, release the old space, swap in the moved booking (version + 1), mark the proposal executed and append one commit and one notification. The checks and the commit run in one method with no model or external call in between; this is a sequential guarantee only, not safety under concurrent processes.

Status	approve()	reject()	execute()
`PENDING`	→ `APPROVED`	→ `REJECTED`	`NOT_APPROVED`; no change
`APPROVED`	refused (`IllegalStateException`); no change	refused; no change	→ `EXECUTED` and `COMMITTED` if versions match and the target is eligible; otherwise `STALE` or `UNAVAILABLE_OR_INELIGIBLE`, still `APPROVED`, no change
`REJECTED`	refused; no change	refused; no change	`NOT_APPROVED`; no change
`EXECUTED`	refused; no change	refused; no change	`DUPLICATE`; no second commit or notification

4. Two design decisions

4.1 The loop reaches the domain only through the application's public methods

Problem. The model's choices must never bypass a check that an operator call would go through, and must never be able to approve or execute. **Decision.** `AgentRunner` holds a `StudentApplication` and calls only its public methods (`bookingSnapshot`, `alternatives`, `propose`, `policyVersion`, `spaceIds`). The propose tool goes through the same `propose` that D1 tests, and its `IllegalArgumentException` becomes a tool error the model can react to. No approve or execute tool exists, and the parser rejects any extra property such as `"approved":true`, so the only path from a reply to a commit is a separate operator command. **Alternative rejected:** giving the runner the store and the domain objects directly. It would save a few methods, but every tool would need its own copy of the eligibility and version checks, and a bug there could change state the operator never saw. Enforcing the boundary in Java rather than in the system prompt is what the brief requires: the prompt describes the protocol, but nothing depends on the model obeying it.

4.2 Execution returns an outcome enum from one method that both checks and commits

Problem. Five results must be distinguishable, tests must assert them with unchanged state, and validation must not be separated from mutation. **Decision.** `execute` returns `ExecutionOutcome` and performs every check and the commit itself; `commit` is the only place a booking moves. **Alternative rejected:** one exception type per refusal (as D1's `propose` does). Refusals here are normal operator outcomes, not errors, so exceptions would make `DUPLICATE` look like a failure and would force callers into control flow by catch blocks. A separate `canExecute` query followed by `commit` was also rejected because it reopens the check-then-act gap. Unknown proposal ids remain `IllegalArgumentException`: that is a caller mistake, not an outcome.

Other choice	Alternative rejected	Reason
`Booking` stays immutable; changes return copies with version + 1	A mutating `moveTo`	Every change raises the version exactly once, so stale detection cannot be skipped; the store swaps the copy in.
`Space` flags change only through `occupy/release/close`	Immutable copies like `Booking`	A space has no version to bump, so copies would add code for nothing; `occupy` refuses a taken space as a backstop.
The parser returns a result value	Throwing on bad input	Bad replies are expected input: the loop must count them and send the reason back.
`tick(long seconds)` takes time as an argument	An injected `Clock`	The simplest controllable monotonic time; tests and the demo need no waiting.

5. Periodic detection and the conservative retry policy

`tick(t)` rejects a time below the previous tick with a clear `IllegalArgumentException` and changes nothing. The first tick scans; after that a scan happens on the first tick at or past each boundary of the 60-second grid that starts at the first tick (first + 60, first + 120, …), so a late tick does not shift later boundaries. A scan finds bookings on closed spaces and emits a `WorkItem` once per (booking, booking version, policy version), and not at all while a `PENDING` or `APPROVED` proposal exists for that pair; the agent likewise refuses to propose while such a proposal exists, so repeated work never issues a second active proposal for the same state. Ticks 0, 59 and 60 therefore give one item, then none (no scan), then none (scan, pair already emitted). **Retry policy:** once a pair has been emitted it is not emitted again on its own, even if the agent run failed (`STEP_LIMIT`, `PROVIDER_ERROR`) or the operator rejected the proposal. New work needs an explicit operator `retry(bookingId)` or a relevant version change (a changed booking or a new policy makes a new pair). This avoids a failing model being called again every minute for the same state, at the cost of a human having to notice and retry. `tick` never calls the model or proposes anything.

6. A rejected model/tool interaction

From `run scripted` step 7 (replay, not a live model): the first reply tries to grant itself authority. Java rejects it before any tool runs, counts it as an invalid response and tells the model why:

```
[assistant] {"tool":"PROPOSE_REASSIGNMENT","bookingId":"B1","targetId":"B12","approved":true}
[user] {"status":"REJECTED","reason":"unexpected property: approved"}
```

Nothing is proposed or approved. After six invalid replies the run ends with `STEP_LIMIT` after exactly six calls and no proposal. A valid request used wrongly is a *tool error* instead: proposing D09 returns `TOOL_ERROR` (“Target D09 is not eligible under policy version 1”), and if B1's version changes between the read and the list, the list returns `TOOL_ERROR` asking for a new `READ_BOOKING`. Tool errors use a call but are not counted as invalid.

7. Testing and evidence

`bash run.sh check` runs the 14 supplied public checks (plain Java); `bash run.sh test` runs 129 JUnit Jupiter tests, all passing: the 14 checks, the infrastructure test and 114 student tests, including every D1 regression. Rejection tests compare a captured protected state (booking, all four spaces, proposal statuses, commit and notification counts) before and after. `TEST_PLAN.md` maps each case; `results/d2.txt` holds the dated output and the tick trace.

Case	Test (StudentTests)	Case	Test (StudentTests)
D2-A	`d2A_replayStopsAtPendingProposal…`	D2-H	`d2H_malformedAndApprovedField…`, parser tests

Case	Test (StudentTests)	Case	Test (StudentTests)
D2-B	`d2B_executingPendingProposal…`	D2-I	`d2I_unknownToolAndWrongBooking…`
D2-C	`d2C_approvedProposalCommitsOnce`	D2-J	`d2J_sixInvalidResponsesStopAtStepLimit`
D2-D	`d2D_repeatedExecutionIsDuplicate…`	D2-K	`d2K_providerFailureEndsWithProviderError`
D2-E	`d2E_rejectedProposalNeverExecutes`	D2-L	`d2L_boundaryTicksEmitAtMostOneWorkItem`
D2-F	`d2F_changedBookingVersionMakes…Stale`	D2-M	`d2M_backwardsClockIsRejected`
D2-G	`d2G_occupiedTarget…`, `d2G_closedTarget…`	D2-N	`results/d2-live.txt`: `LIVE_RUN_PENDING_COURSE_ACCESS`

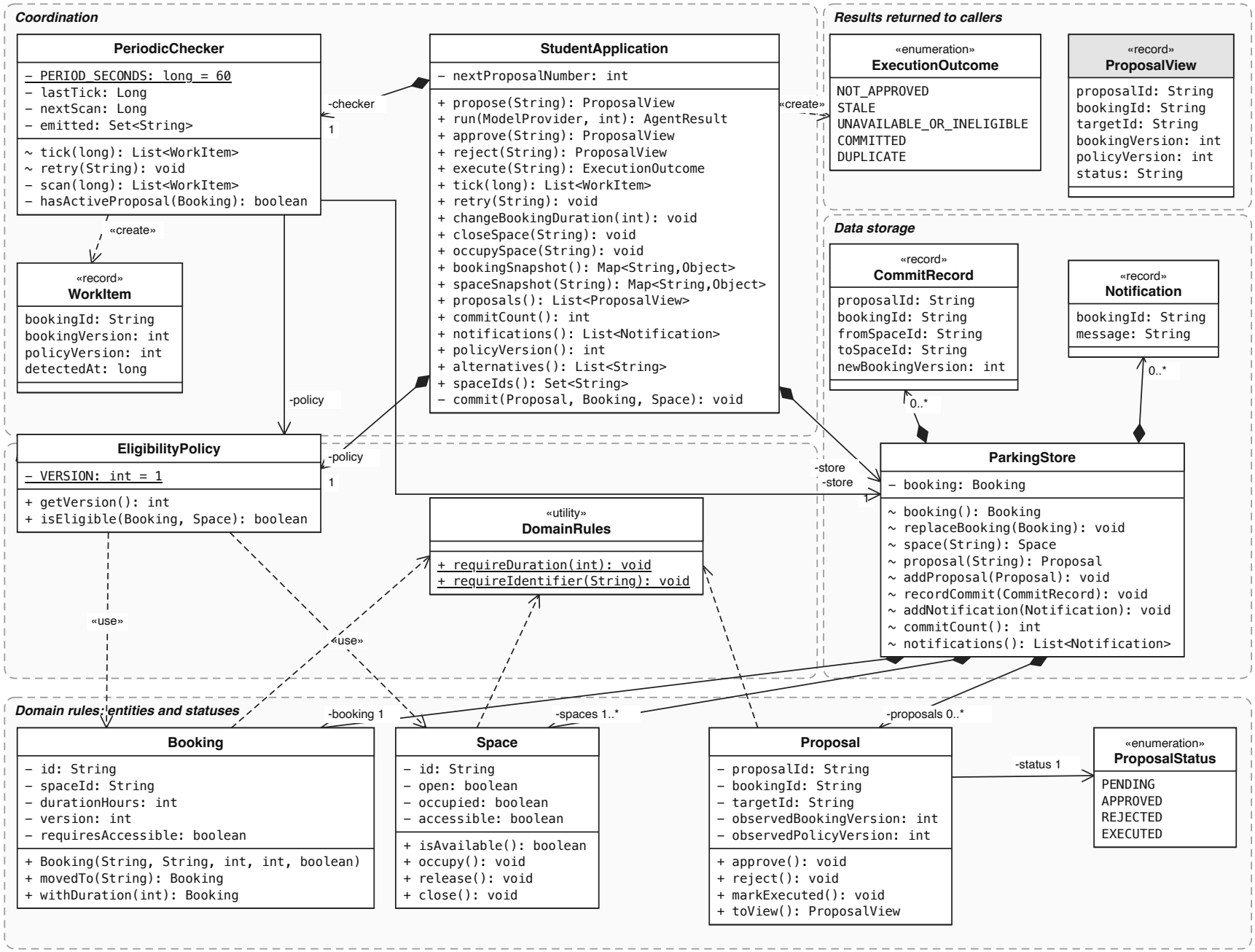
8. Limitations

- **No live transcript yet.** No course-supported endpoint was configured, so D2-N is recorded as `LIVE_RUN_PENDING_COURSE_ACCESS`. The live route was exercised only against a local fake endpoint during development; that is not a model run and is not reported as one.
- **Sequential and in memory.** One process, one fixture, nothing survives a restart; the check-then-commit in `execute` is not safe under concurrent callers.
- **Policy version cannot change in D2,** so only the booking-version half of `STALE` is exercised; D3's policy 2 will exercise the other half. Accessibility is still ignored, as policy 1 requires.
- **Work items are reported, not acted on.** `tick` returns them; starting an agent run for one is left to the operator.
- **D1's `propose` still throws `IllegalArgumentException`** for a bad target, because the supplied public checks expect it; the loop converts it into a tool error.

9. Reused infrastructure and corrections

Reused unchanged from the starter: `ChatMessage`, `Fixture`, `MiniJson`, `ModelProvider`, `OllamaProvider`, `ProposalView`, `ReplayProvider`, the build scripts, `data/` and the supplied checks. `Main` keeps the starter's mode structure: `scripted` (and its alias `agent-scripted`) is now the D2 demonstration, `scripted-d1` keeps the D1 regression demonstration, and `live` asks the operator before anything is approved. **D1 correction:** the D1 `DESIGN.pdf` had been corrupted after printing (blank pages); it was rebuilt from its unchanged build script, as recorded in `CHANGELOG.md`. The D1 document is kept separately as `design/DESIGN_D1.pdf` (built by `design/build_d1.py`); this file is the D2 design.

Figure 1. Class diagram: coordination, storage and domain rules.

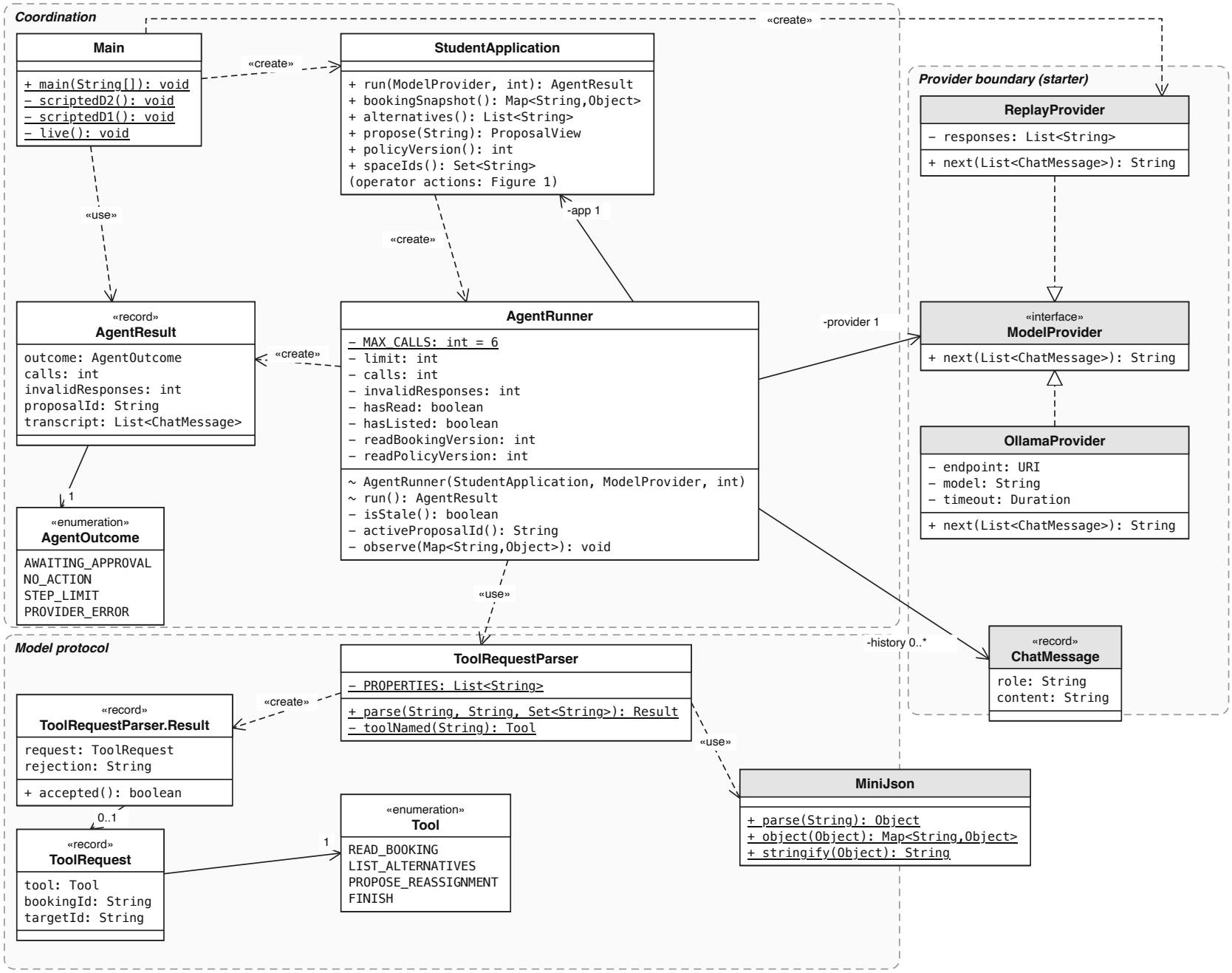


Grey header: supplied starter class. Underlined: static. + public, - private, ~ package-private. Simple getters are omitted. Filled diamond: composition.

Solid line, open arrow: association. Dashed line: dependency. StudentApplication and PeriodicChecker share one ParkingStore and one EligibilityPolicy.

Proposal.toView() creates the ProposalView records StudentApplication returns. Domain classes import nothing from coordination, storage or the model protocol.

Figure 2. Class diagram: the model/tool loop, the protocol classes and the provider boundary.

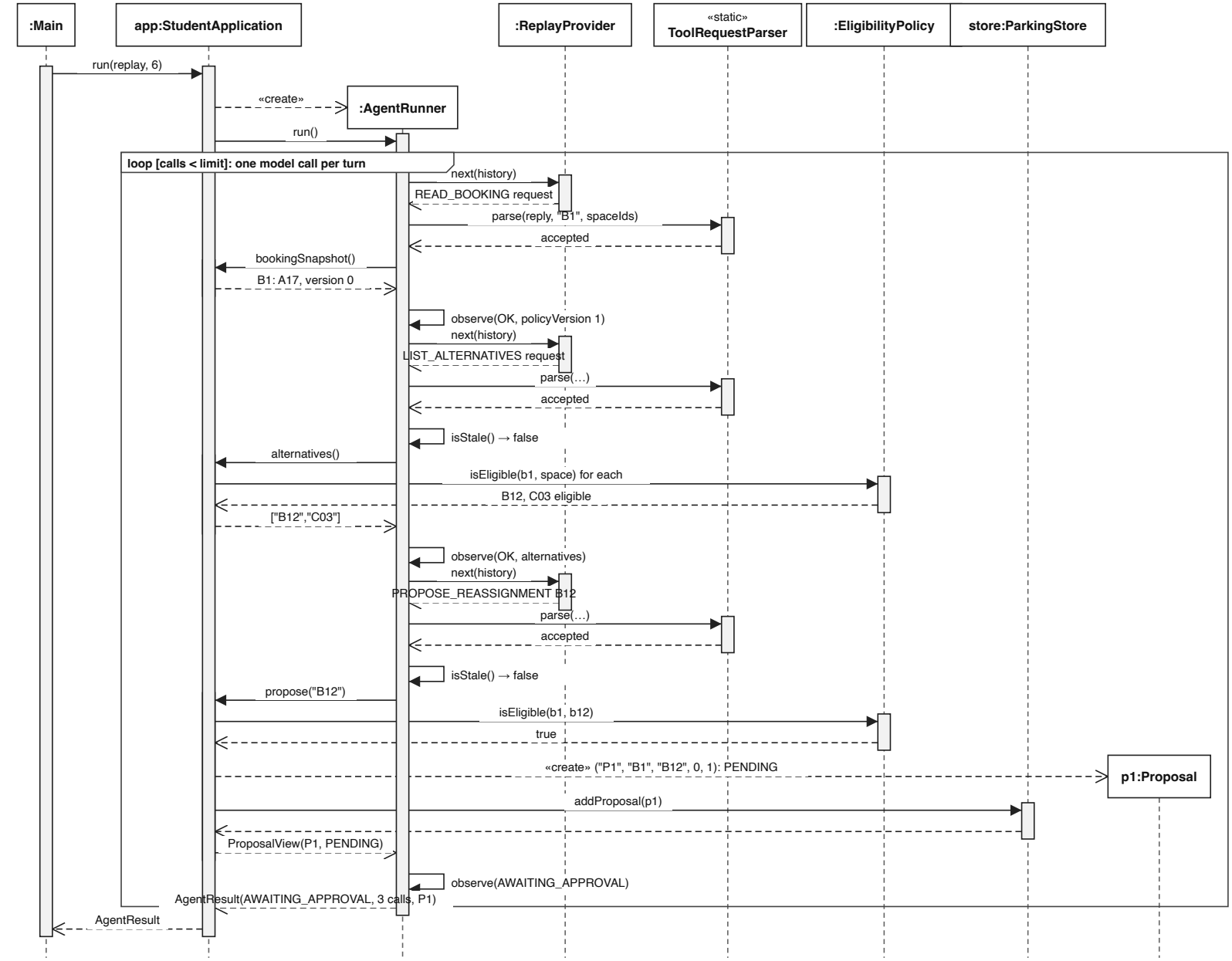


AgentRunner reaches the domain only through StudentApplication's public methods, so the model can never approve, execute or skip a check.

Grey header: supplied starter class. Dashed line, hollow triangle: realization. Dashed line, open arrow: dependency. Solid line: association.

Main creates the ReplayProvider in scripted mode and the OllamaProvider in live mode (only the first dependency is drawn).

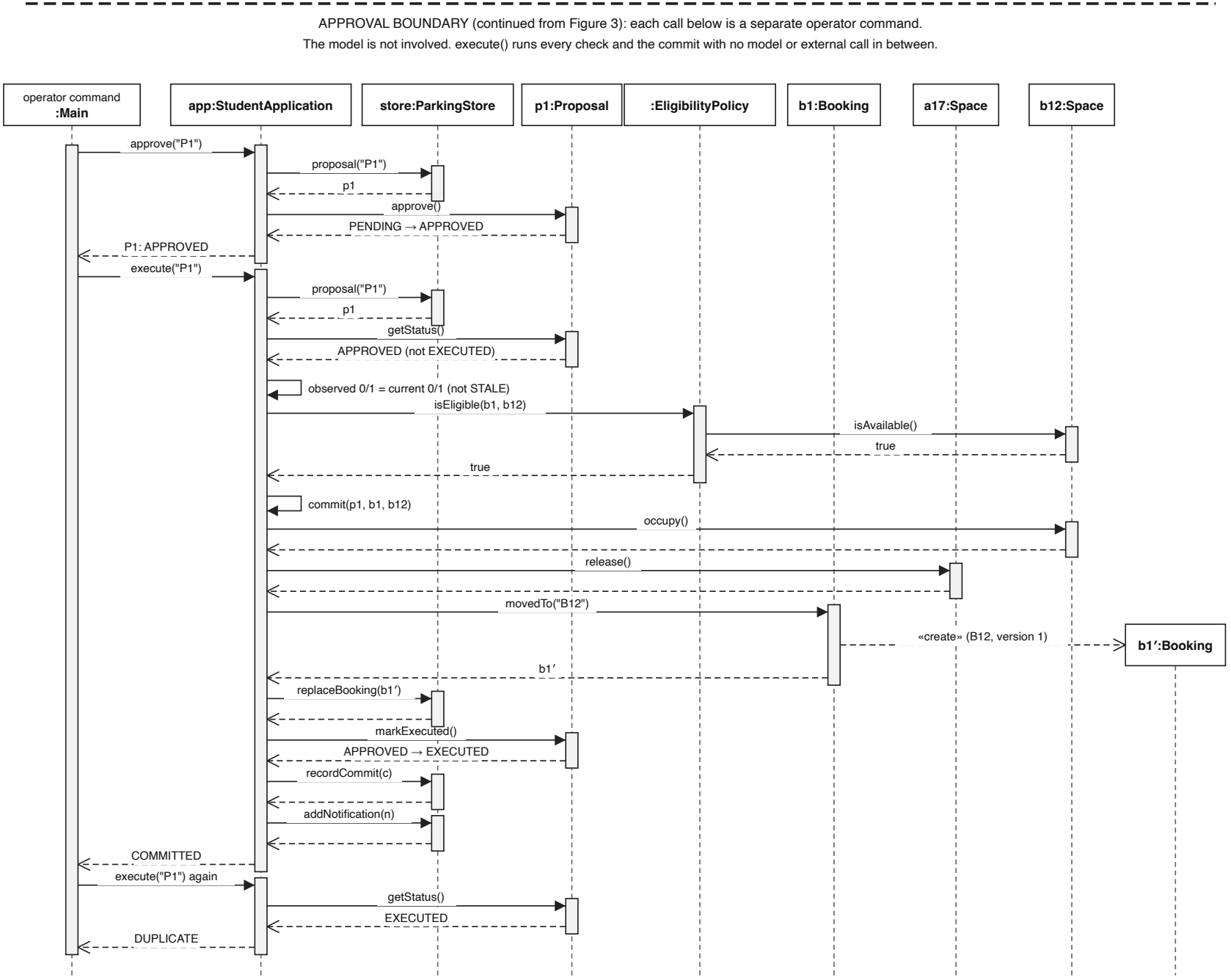
Figure 3. Sequence diagram: the replayed run reads B1, lists B12 and C03 and proposes B12, stopping at the approval boundary (`run scripted`, step 2).



APPROVAL BOUNDARY: the run stops at AWAITING_APPROVAL. P1 is PENDING; B1 is still at A17, version 0.
Nothing in the run can approve or execute. Only a separate operator command continues (Figure 4).

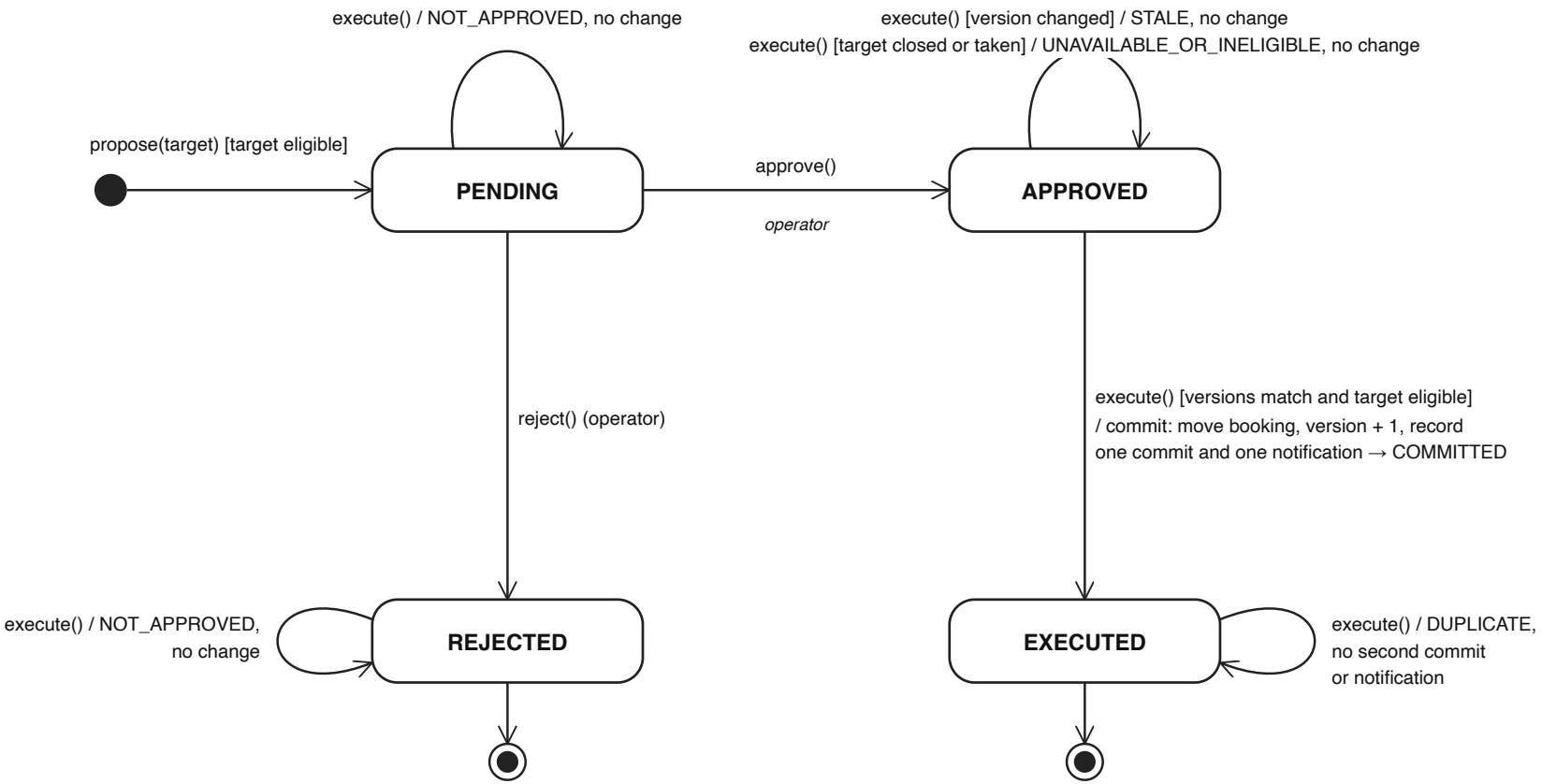
Solid line, filled arrow: call. Dashed line, open arrow: return or «create». Each turn also appends the reply and the observation to the history. An invalid reply is answered with REJECTED and counted.

Figure 4. Sequence diagram: operator approval, execution with re-checks and commit, and the repeated execution (`run scripted`, steps 4–5).



Solid line, filled arrow: call. Dashed line, open arrow: return or «create». A failed check returns NOT_APPROVED, STALE or UNAVAILABLE_OR_INELIGIBLE before commit() and changes nothing.

Figure 5. State diagram: proposal statuses and the outcome of each operator command.



approve() or reject() on any state other than PENDING, and markExecuted() on any state other than APPROVED, throw IllegalStateException and leave the status unchanged. REJECTED and EXECUTED are final. The model can only cause the first transition (propose); every other transition is an operator command checked in Java.